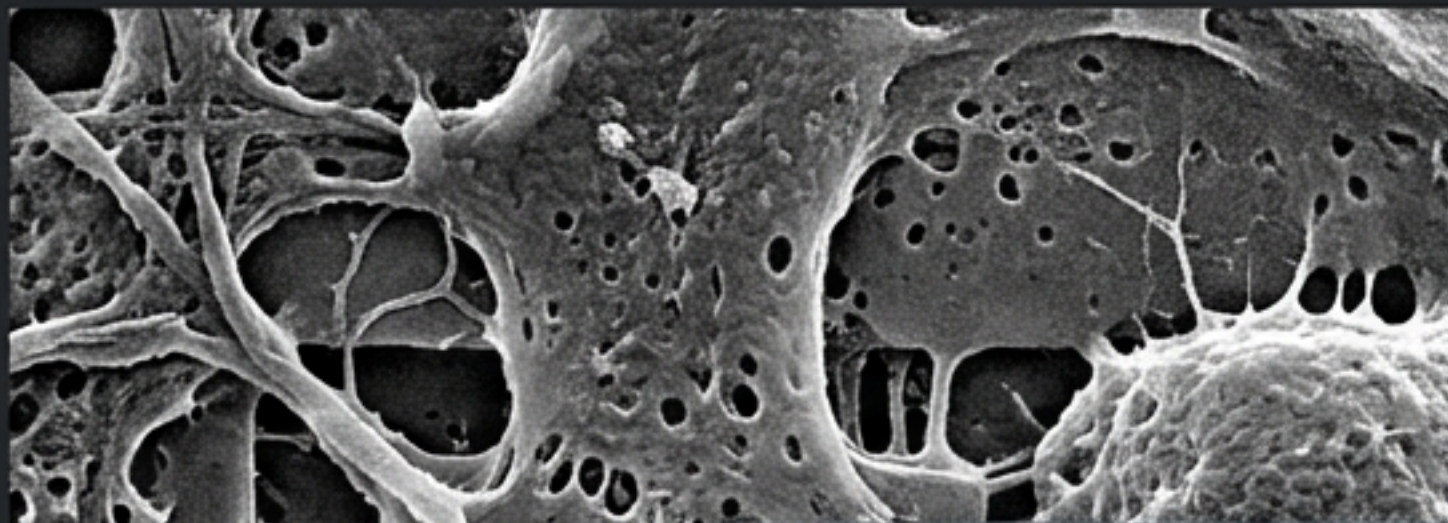
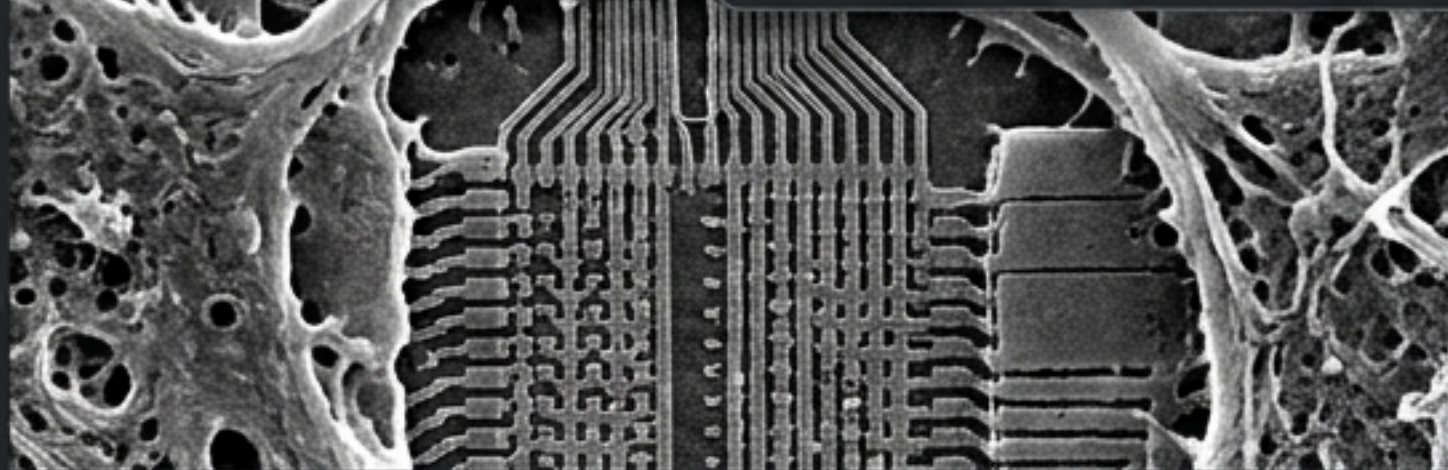




Procesamiento Digital de Imágenes: De la Matemática al Código

Transformaciones de Intensidad y Filtrado Espacial en Python



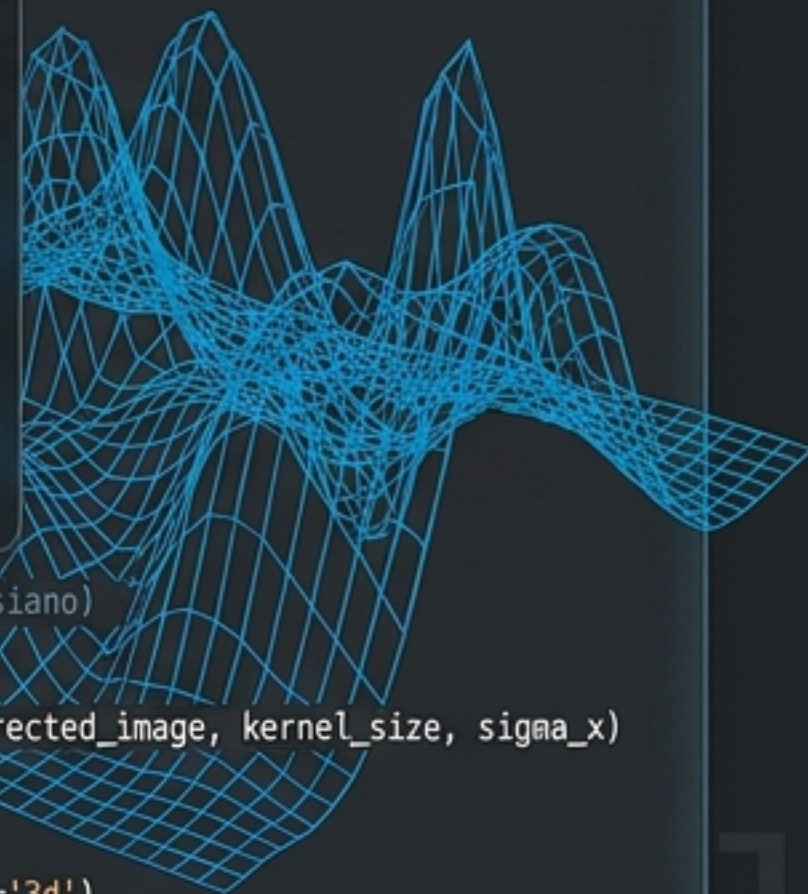
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Cargar la imagen en escala de grises
image = cv2.imread('bio_sensor_raw.png', cv2.IMREAD_GRAYSCALE)

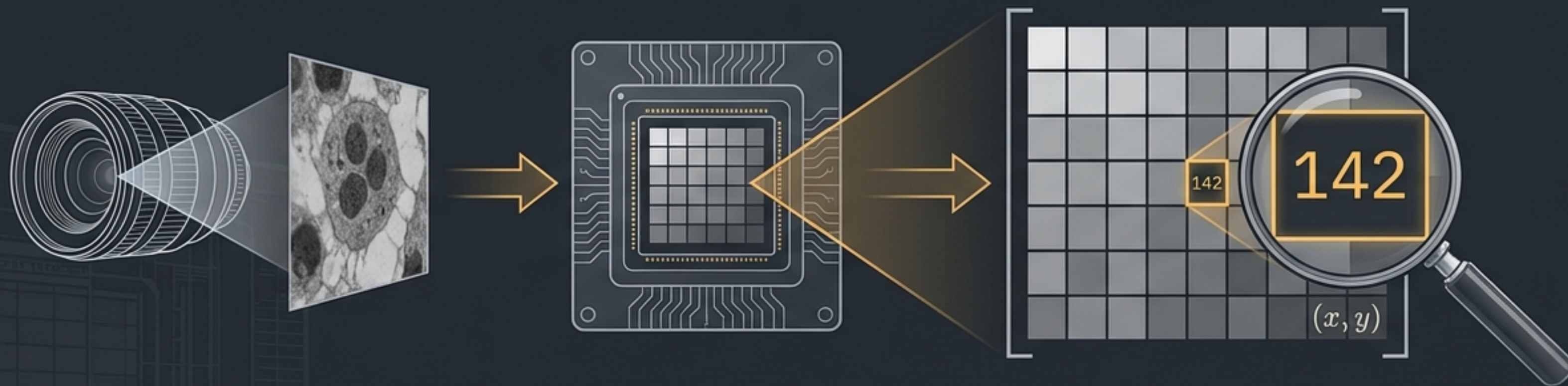
# Transformación de Intensidad (Corrección Gamma)
gamma = 1.5
corrected_image = np.power(image / 255.0, gamma)
corrected_image = np.uint8(corrected_image * 255)
```

```
# Filtrado Espacial (Desenfoque Gaussiano)
kernel_size = (5, 5)
sigma_x = 1.0
blurred_image = cv2.GaussianBlur(corrected_image, kernel_size, sigma_x)

# Visualización de la superficie 3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
X, Y = np.meshgrid(np.arange(image.shape[1]), np.arange(image.shape[0]))
ax.plot_wireframe(X, Y, image, color='cerulean')
plt.show()
```



El Lienzo Computacional: La Imagen como Función $f(x, y)$

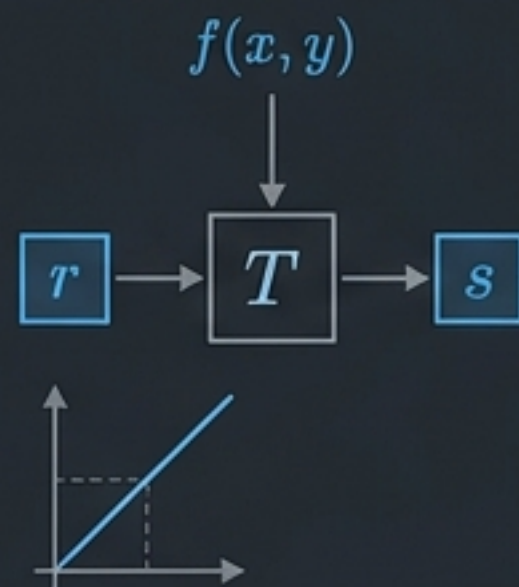


Dominio de Intensidad (Procesamiento Puntual)

Depende exclusivamente del valor del píxel actual.

Mapeo directo: Modifica el contraste y la luminosidad sin alterar la geometría.

$$s = T(r)$$

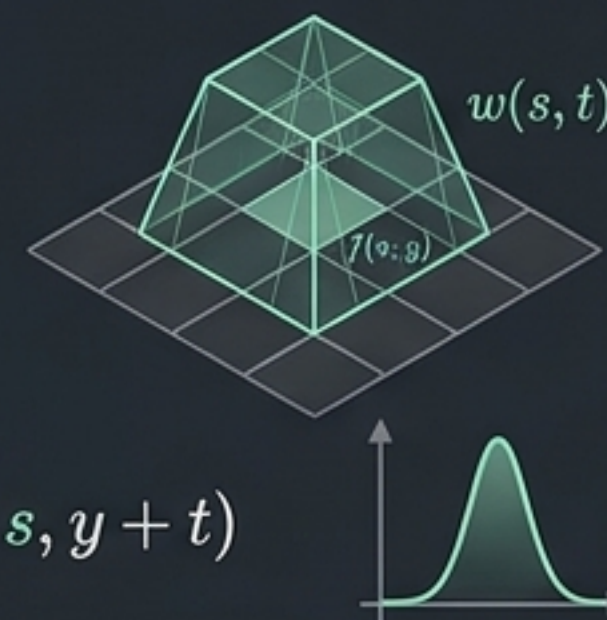


Dominio Espacial (Procesamiento Vecinal)

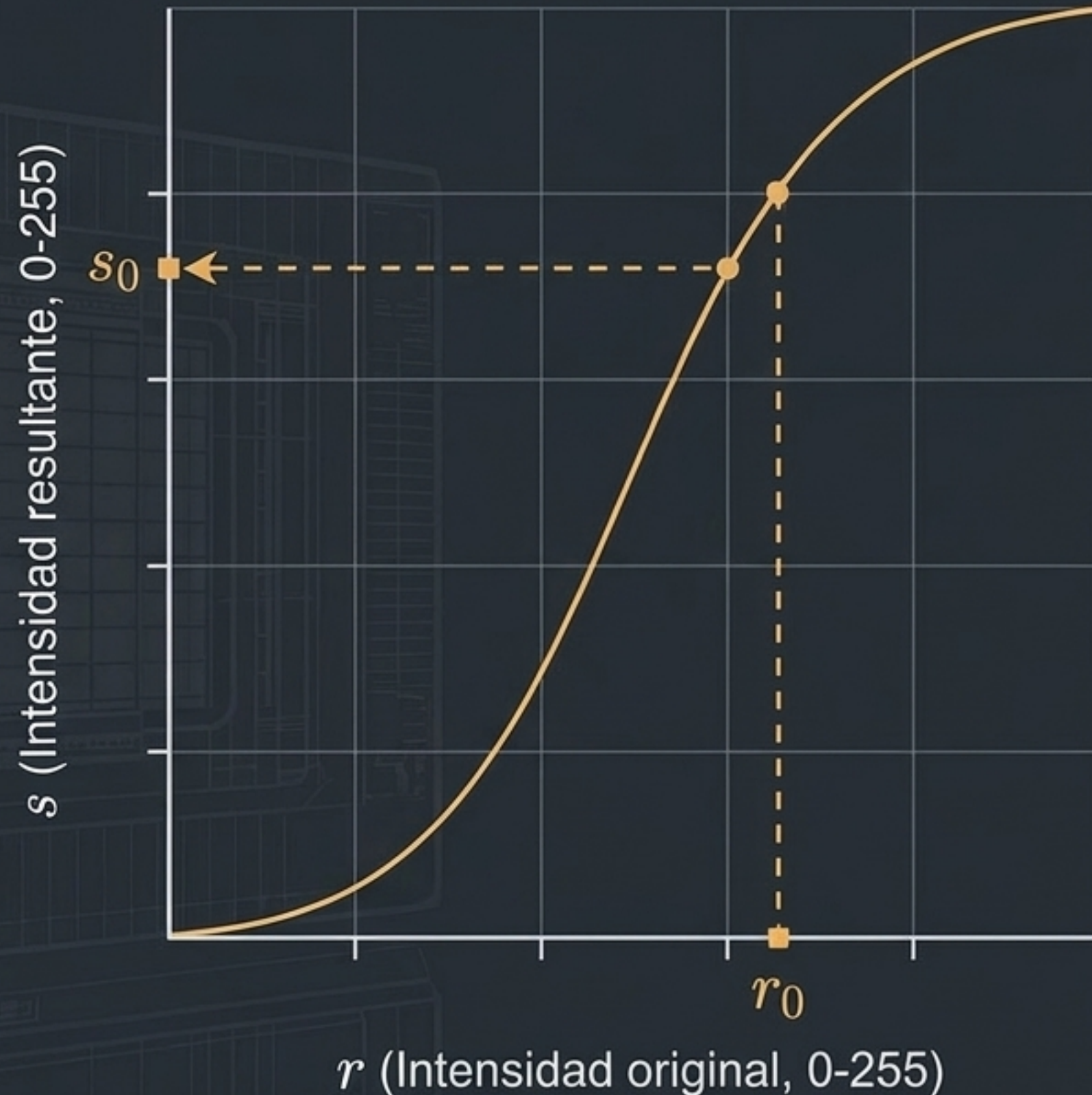
Depende de los **píxeles adyacentes** (vecindad).

Calcula **correlaciones locales**:
Suaviza ruidos o detecta bordes.

$$g(x, y) = \sum_x \sum_t w(s, t) f(x + s, y + t)$$



Acto I: Transformaciones de Intensidad y el Mapeo $s = T(r)$



Una transformación de intensidad opera bajo una regla estricta: el píxel cambia su valor, pero nunca su posición.

r : Nivel de gris de entrada (0 a $L-1$, típicamente 255).

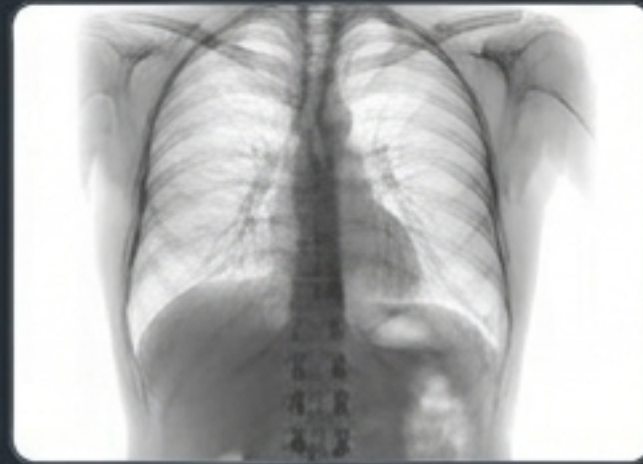
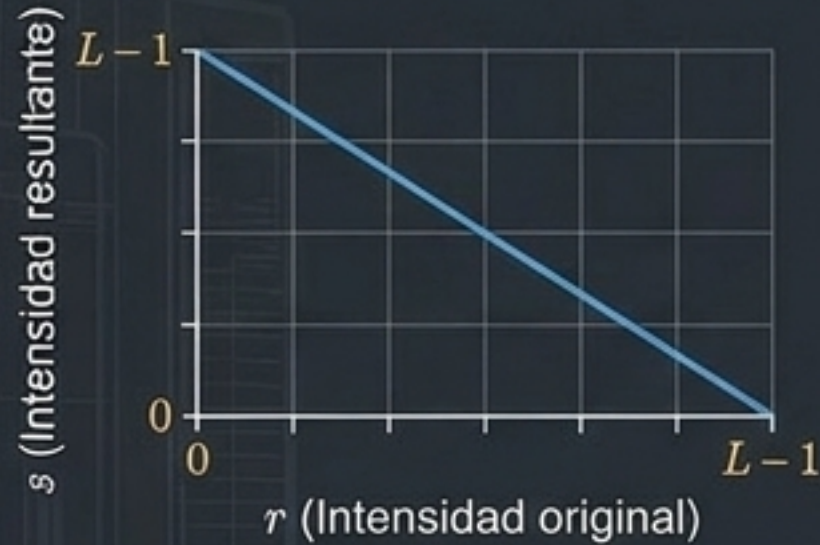
s : Nivel de gris de salida.

T : La función de transformación matemática que define el nuevo mapeo tonal.

Transformaciones Básicas: Negativo y Logaritmo

El Negativo

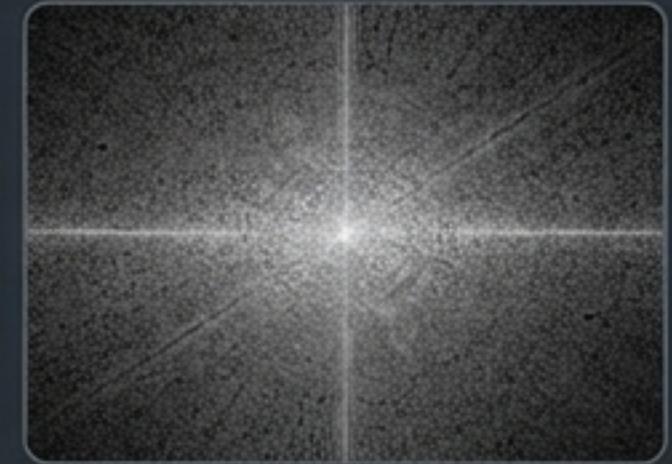
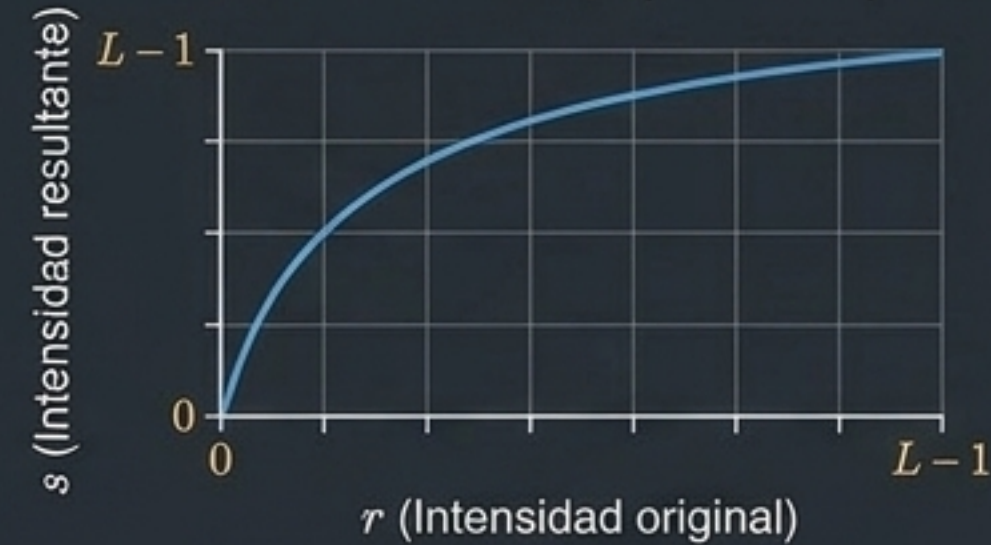
$$s = L - 1 - r$$



Impacto Visual: Realza detalles blancos o grises ocultos sobre fondos masivamente oscuros (ej. radiografías).

El Logaritmo

$$s = c \cdot \log(1 + r)$$

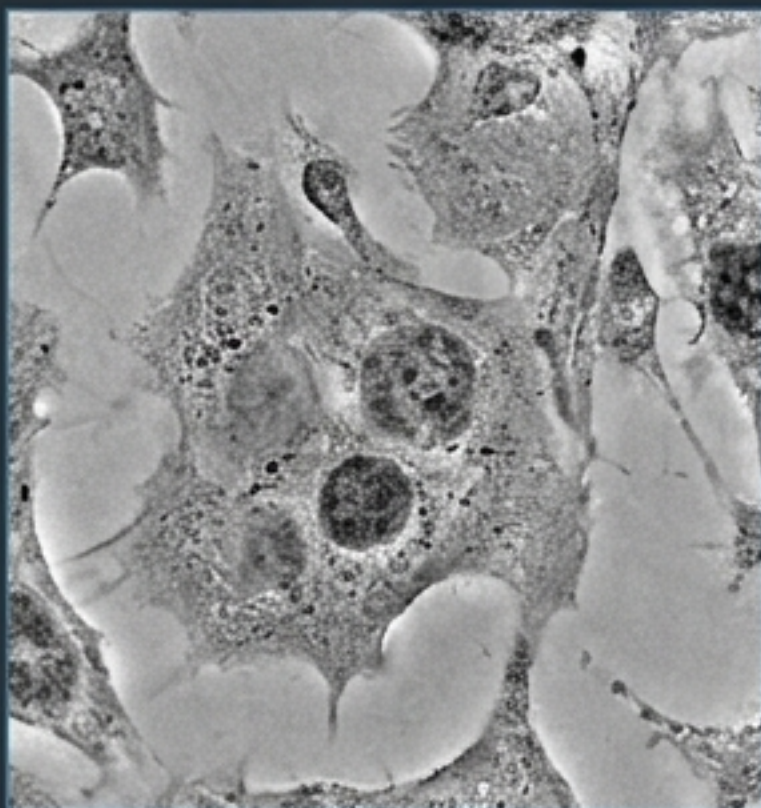


Impacto Visual: Expande los píxeles oscuros y comprime los brillantes. Esencial para visualizar espectros de Fourier de alto rango dinámico.

Transformación de Potencia: Corrección Gamma

$$s = c \cdot r^\gamma$$

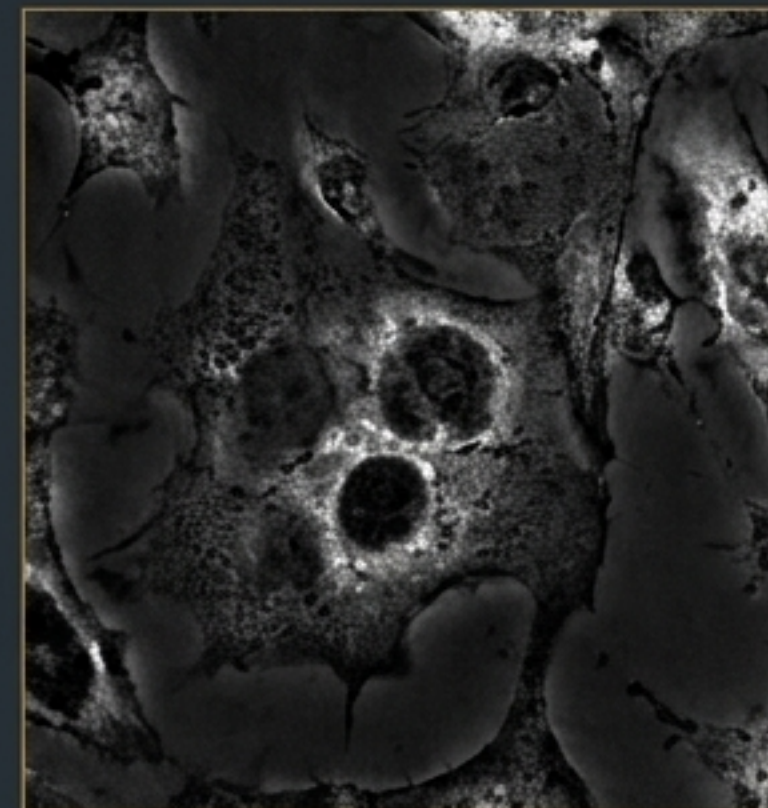
$\gamma = 0.5$



Expansión Oscura ($\gamma < 1$):

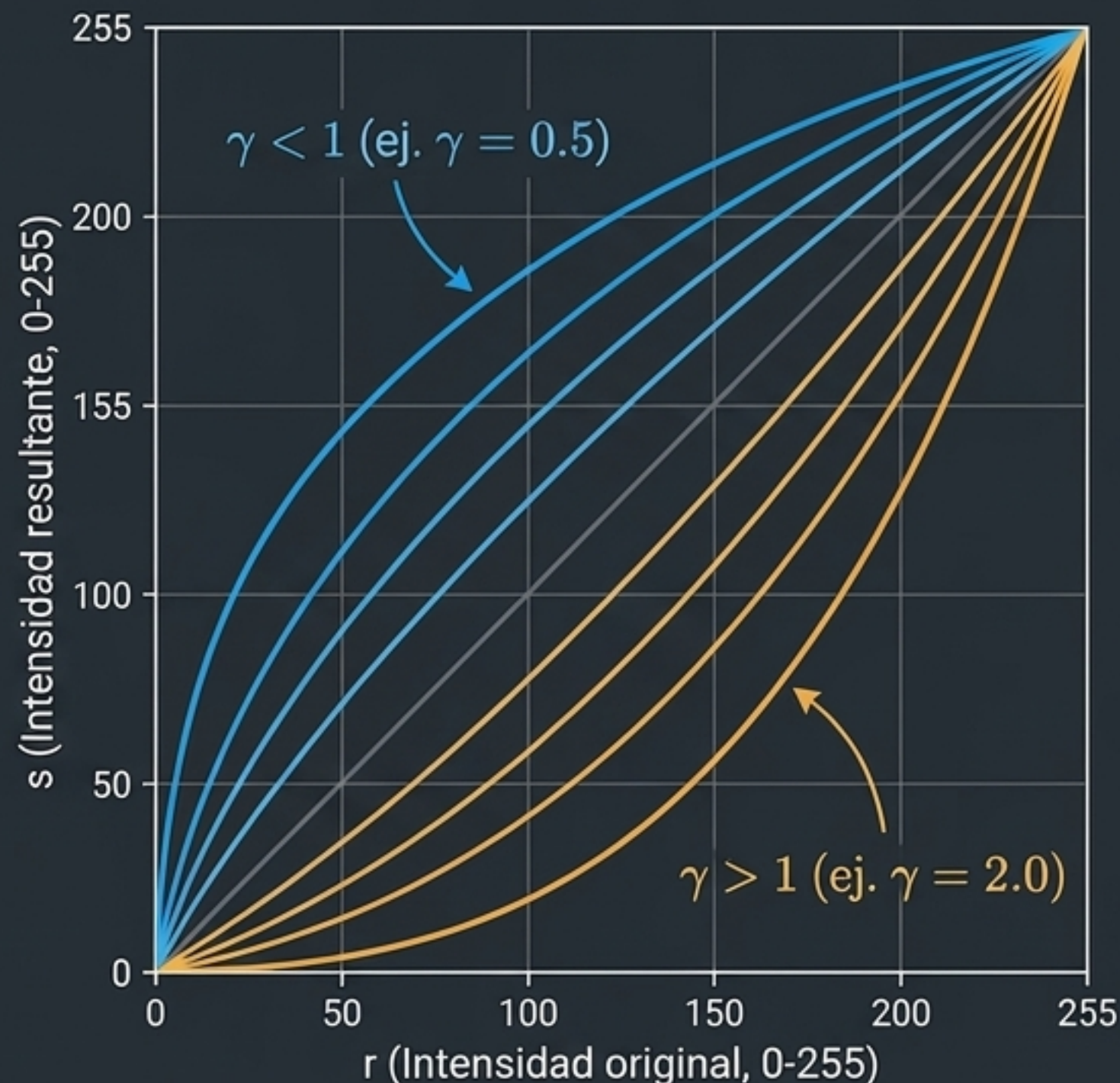
Estira el rango dinámico en las regiones oscuras. Aclara la imagen, revelando texturas ocultas.

$\gamma = 2.0$

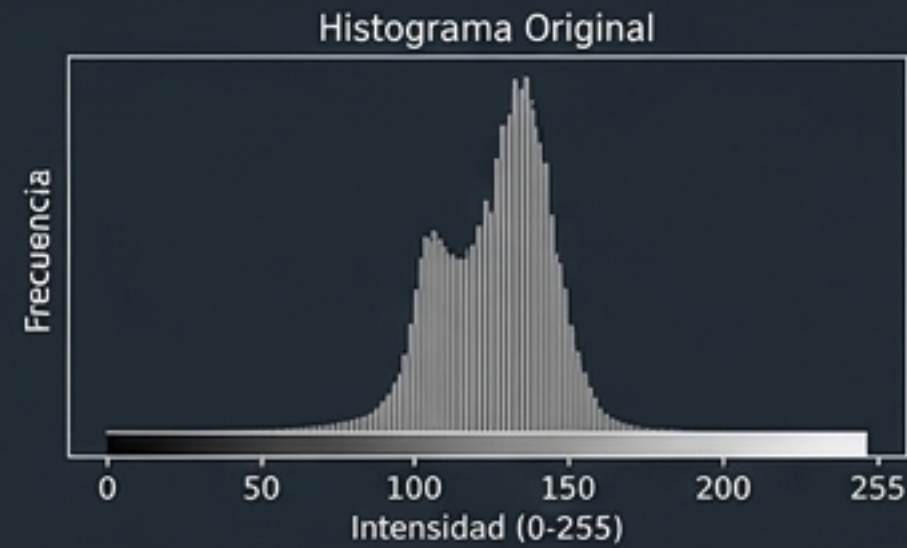


Compresión Oscura ($\gamma > 1$):

Comprime los niveles bajos y expande los altos. Oscurece la imagen, ideal para eliminar ruido de fondo.



Ecualización de Histograma: Maximizando la Entropía



El Problema:

Imágenes con contraste deficiente concentran sus píxeles en una banda estrecha. El rango dinámico utilizado es mínimo.

$$s = T(r) = \int_0^r p_r(w)dw$$

La Solución Matemática

Utilizar la Función de Distribución Acumulada (CDF) como transformación $T(r)$. Esto mapea la distribución acumulativa original a una distribución uniforme ideal.



El Resultado: Distribución uniforme de niveles de gris que maximiza el rango dinámico global $[0, L-1]$. Mejora significativa en el contraste y la visibilidad de los detalles.

Implementación en Python: Dominio de Intensidad

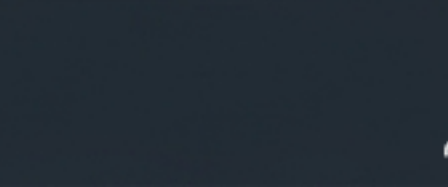
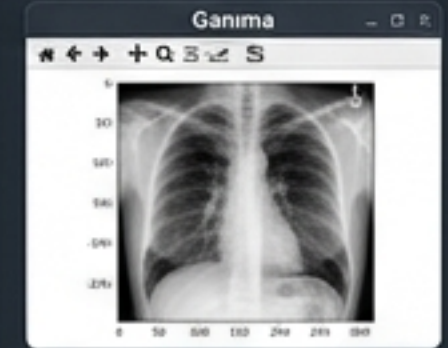
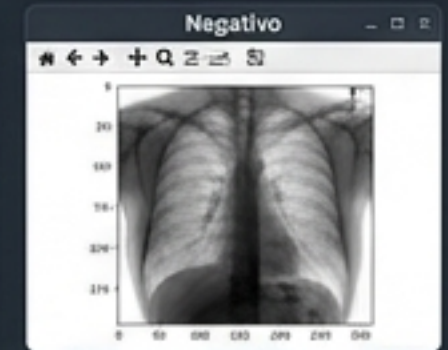
```
1  # Negativo
2  s = cv2.bitwise_not(img)
3
4  # Logaritmo
5  s = np.uint8(c * np.log1p(img))
6
7
8  # Gamma (con Look-Up Table)
9  LUT = np.array([(i/255.0)**gamma)*255 for i in np.arange(0, 256)]).astype('uint8')
10 s = cv2.LUT(img, LUT)
11
12 # Ecualización
13 s = cv2.equalizeHist(img)
14
15 # Visualization
16 plt.imshow(s, cmap='gray'); plt.show()
```

Negativo

Logaritmo

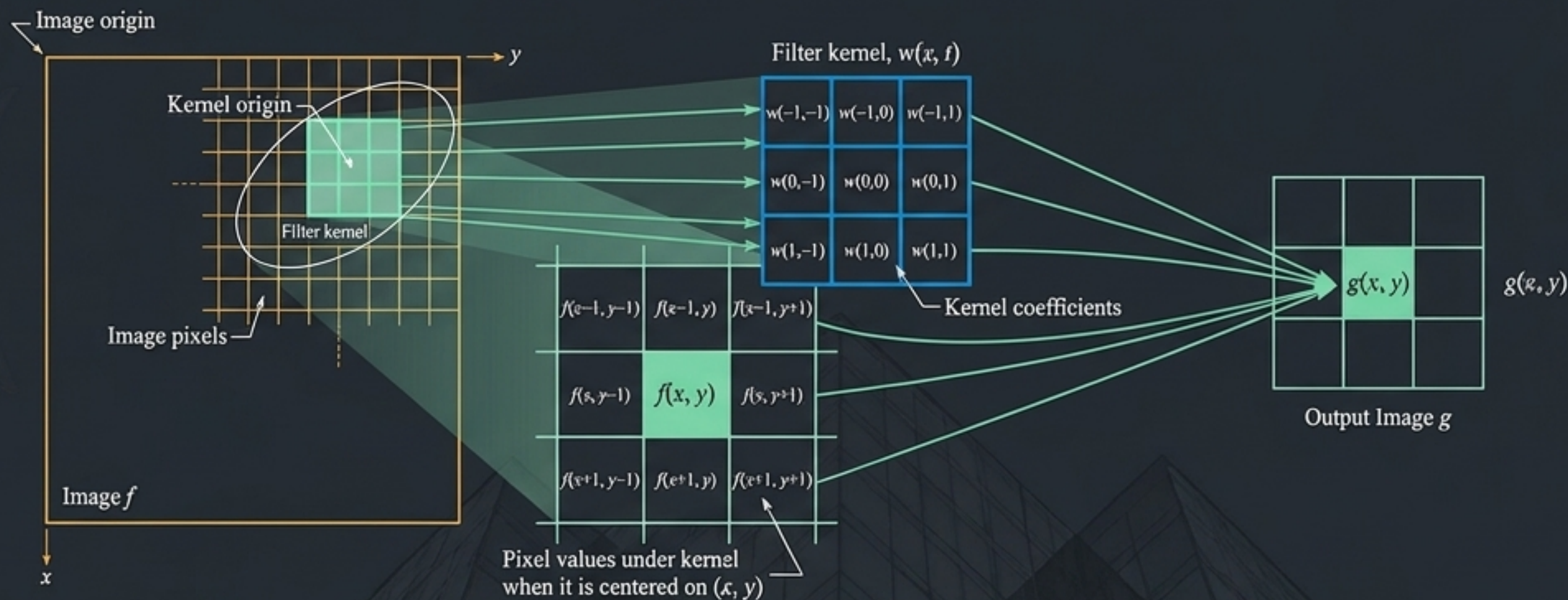
Gamma (con Look-Up Table)

Ecualización



Acto II: Filtrado Espacial y el Motor de Convolución 2D

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$



El procesamiento trasciende el píxel individual; ahora el valor resultante depende del promedio ponderado de su vecindad.

El **Núcleo (Kernel/Máscara)**: Una matriz de coeficientes que se desliza sistemáticamente sobre la imagen, dictando la naturaleza del filtro.

Filtros Suavizantes (Pasa-bajas): Atenuando la Frecuencia

Regla de Oro: La suma de todos los coeficientes debe ser igual a 1 para no alterar el brillo global de la imagen.

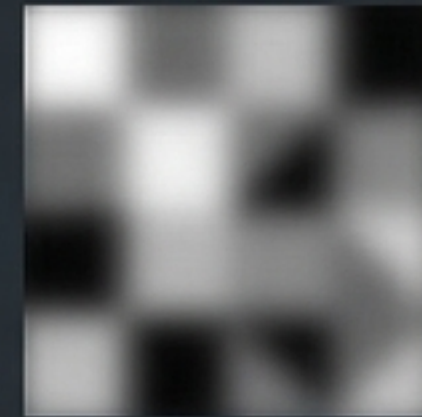
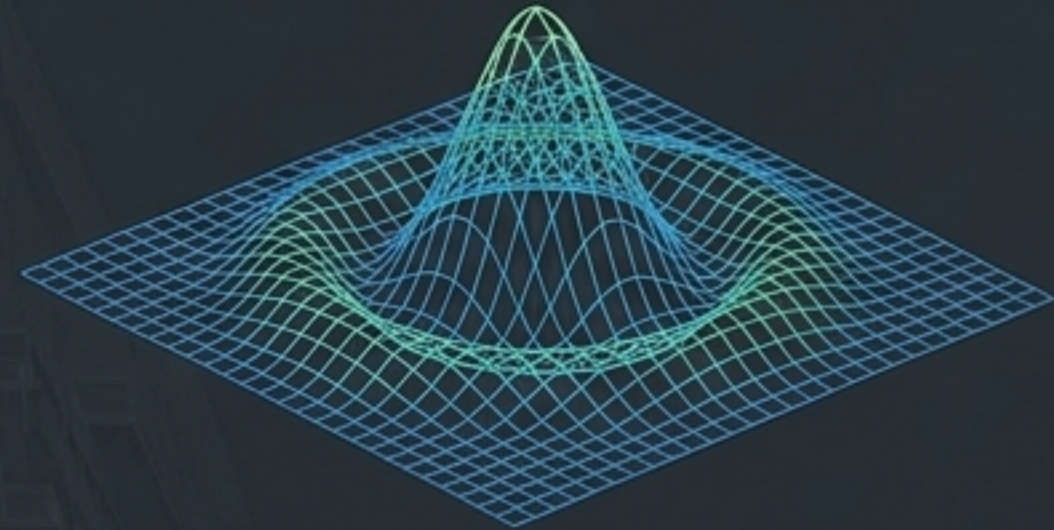
Filtro Promediador (Box)

$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Reduce transiciones bruscas y ruido aleatorio, pero destruye la nitidez de los bordes estructurados.

Filtro Gaussiano



Desenfoca de manera ponderada (prioridad al píxel central). Preserva mejor la estructura macro que el promediador simple.

La Derivada Espacial: Detección de Bordos (Primer Orden)

Regla de Oro: La suma de los coeficientes de un filtro de gradiente siempre es 0. Áreas de intensidad constante se cancelan mutuamente, resultando en negro.

Intensidad ($f(x)$)



Derivada ($f'(x)$)



Los **bordes** son **altas frecuencias** detectadas mediante derivadas.

Matrices de Gradiente (Prewitt & Sobel)

Prewitt Operator

$$G_x: \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y: \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Sobel Operator

$$G_x: \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y: \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Operador de Sobel: Aproxima derivadas parciales en los ejes X e Y mediante convolución discreta.

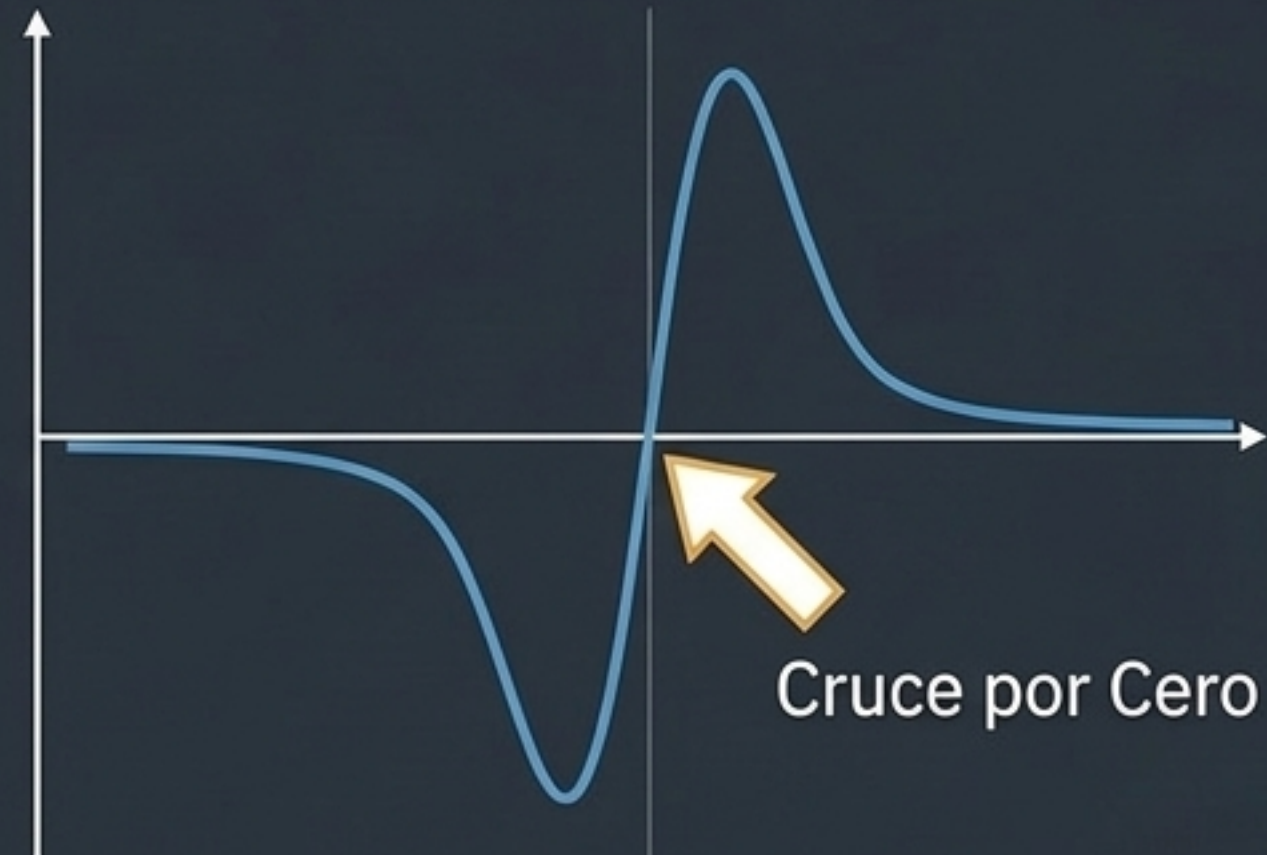


Sensibilidad Extrema: El Laplaciano (Segundo Orden)

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

0	-1	0
-1	4	-1
0	-1	0

A diferencia de Sobel, el Laplaciano detecta variaciones mucho más finas. El "Cruce por Cero" localiza con precisión sub-píxel el centro exacto del borde.



Advertencia: Altamente sensible al ruido impulsivo, por lo que requiere suavizado previo (Laplaciano del Gaussiano / LoG).

Implementación en Python: Motores de Convolución

Filtro Promediador (Box)

```
blur = cv2.blur(img, (5,5))
```

Filtro Gaussiano

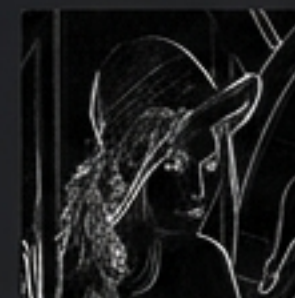
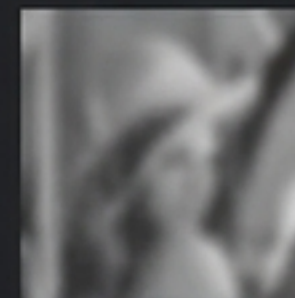
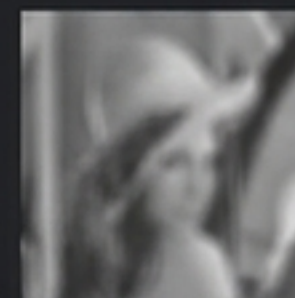
```
gauss = cv2.GaussianBlur(img, (5,5), sigmaX=0)
```

Gradiente Sobel (Eje X)

```
sobel_x = cv2.Sobel(img, cv2.CV_64F, dx=1, dy=0, ksize=3)
```

Laplaciano

```
laplacian = cv2.Laplacian(img, cv2.CV_64F)
```



Matriz de Síntesis: Anatomía del Filtrado Espacial

Categoría	Suma de Coeficientes	Dominio de Frecuencia	Impacto Visual	Algoritmos Clave
Suavizado (Pasa-bajas)	$\Sigma = 1$	Suprime Altas Frecuencias	Difumina ruido, borra bordes finos	Box (Promedio), Gaussiano, Mediana
Realce / Bordes (Pasa-altas)	$\Sigma = 0$	Pasa Altas Frecuencias	Extrae geometría, fondo liso se vuelve negro	Sobel, Prewitt, Laplaciano, Canny

Aplicación Práctica: Pipeline de Unsharp Masking (High-Boost)

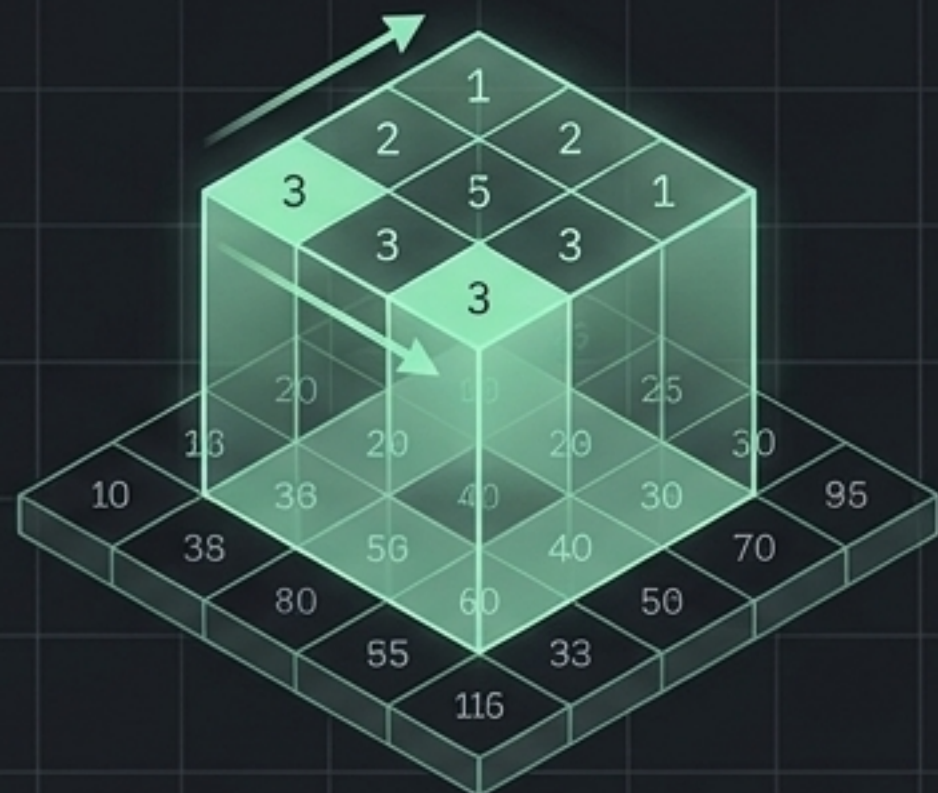
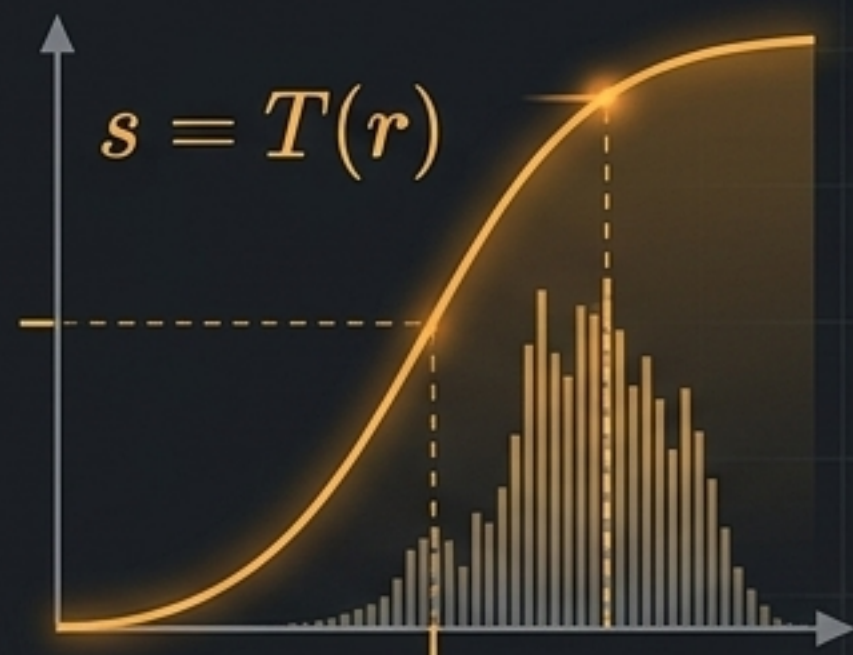


El enmascaramiento de enfoque une ambos dominios computacionales.

$$f_{high-boost}(x, y) = A \cdot f(x, y) - \nabla^2 f(x, y)$$

Utilizado extensivamente en microscopía médica y astronomía para aislar estructuras sin destruir los gradientes de luz originales.

Conclusión: Dominando el Lienzo Computacional



```
cv2 [import cv2]
[ np.array([[ 0,  5,  0],
            [15, 10, 15],
            [20, 20, 30],
            [30, 30, 40]]) ]
[ n.enrollies(n, cv2)]
```

numpy

El **Píxel (Intensidad)**: Modificando la **luz** y el **contraste** mediante matemáticas directas y análisis estadístico.

La **Vecindad (Espacial)**: Modificando la **geometría** y **estructura** mediante cálculo matricial y derivadas.

La **matemática** dicta el comportamiento teórico; **Python** y **OpenCV** ejecutan la visión. ¿Preguntas?